

Experiment No: 1

Aim:- Implement Depth First Search algorithm and Breadth First Search algorithm. Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

Operating System Recommended:- 64-bit Windows OS and Linux

Theory:-

Depth First Search

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

The data structure which is being used in DFS is stack. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.

Depth first Search or Depth first traversal is a recursive algorithm for searching all the vertices of a graph or tree data structure. Traversal means visiting all the nodes of a graph.

Depth First Search Algorithm

A standard DFS implementation puts each vertex of the graph into one of two categories:

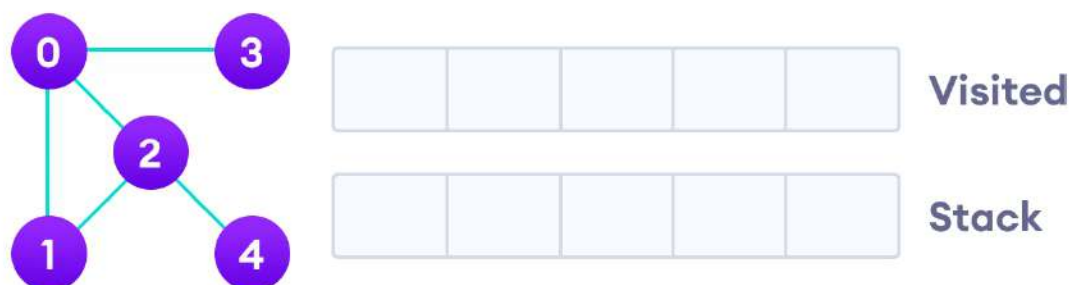
1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles. The DFS algorithm works as follows:

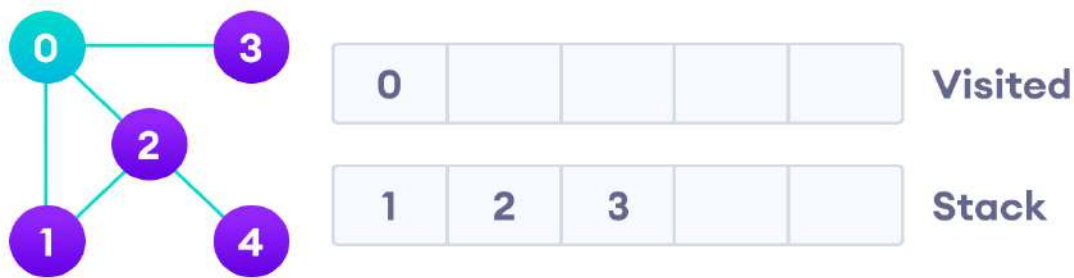
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

Depth First Search Example

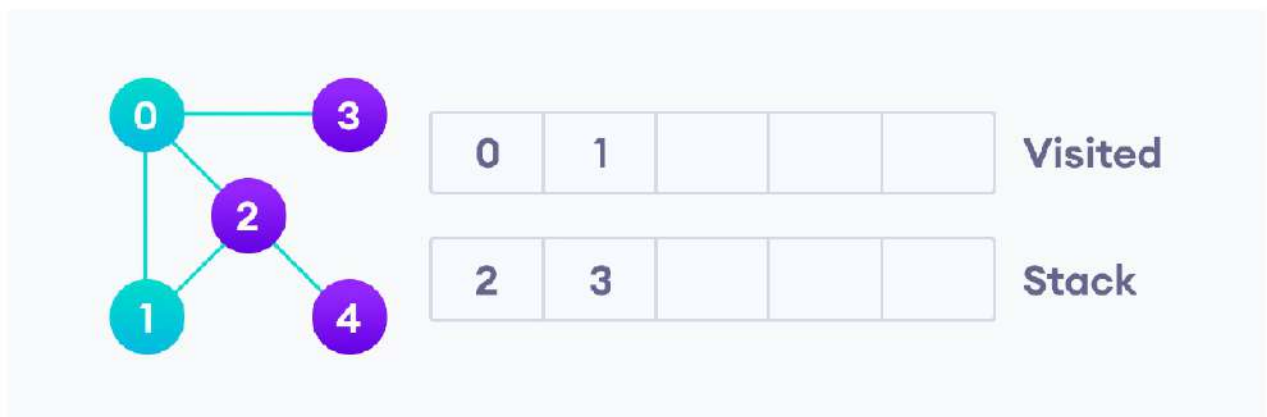
Let's see how the Depth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



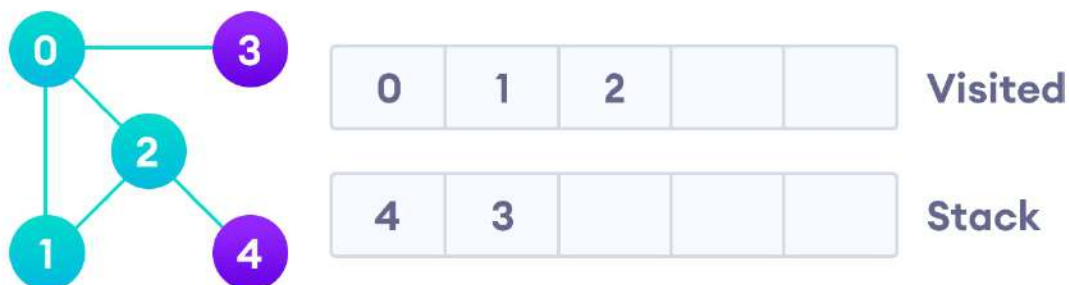
We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.



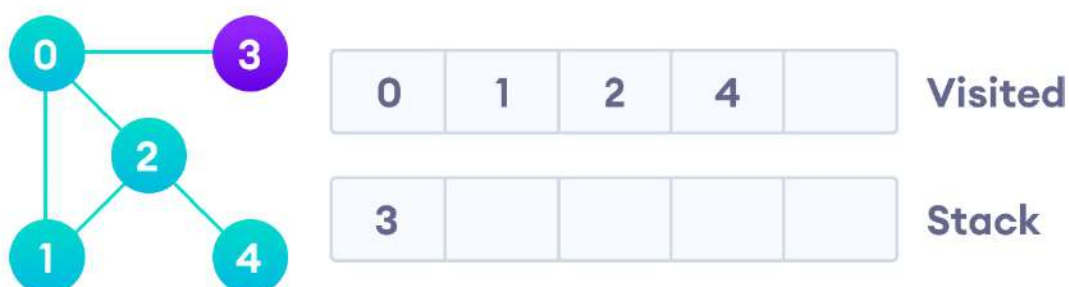
Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

DFS Pseudocode (recursive implementation)

The pseudocode for DFS is shown below. In the `init()` function, notice that we run the DFS function on every node. This is because the graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the DFS algorithm on every node.

```
DFS(G, u)
    u.visited = true

    for each v ∈ G.adj[u]
        if v.visited == false
            DFS(G,v)

init() {
    for each u ∈ G
        u.visited = false

    for each u ∈ G
        DFS(G, u)
}
```

Complexity of Depth First Search

The time complexity of the DFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is $O(V)$.

Breadth First Search

Traversal means visiting all the nodes of a graph. Breadth First Traversal or Breadth First Search is a recursive algorithm for searching all the vertices of a graph or tree data structure.

BFS Algorithm

A standard BFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

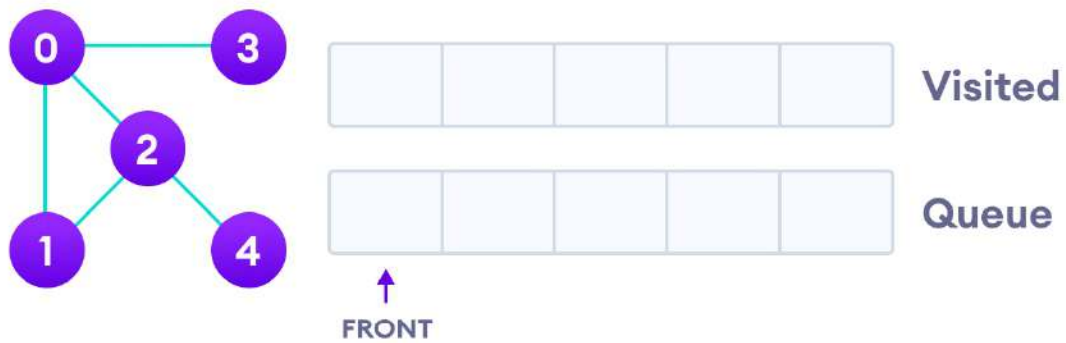
The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The algorithm works as follows:

1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

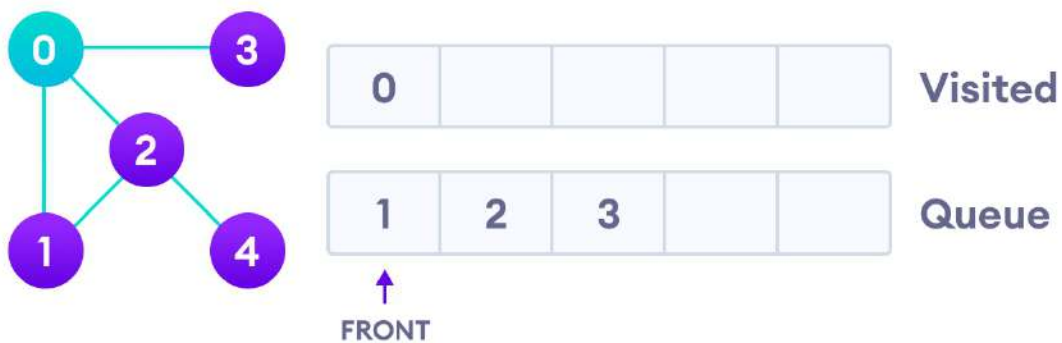
BFS Example

Let's see how the Breadth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



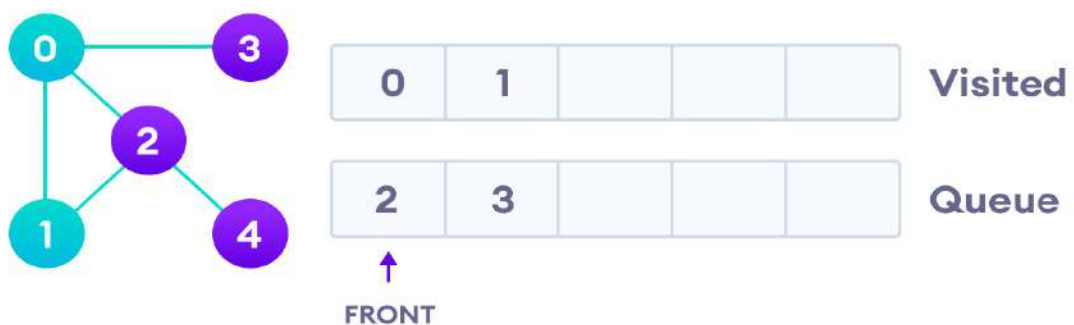
Undirected graph with 5 vertices.

We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.



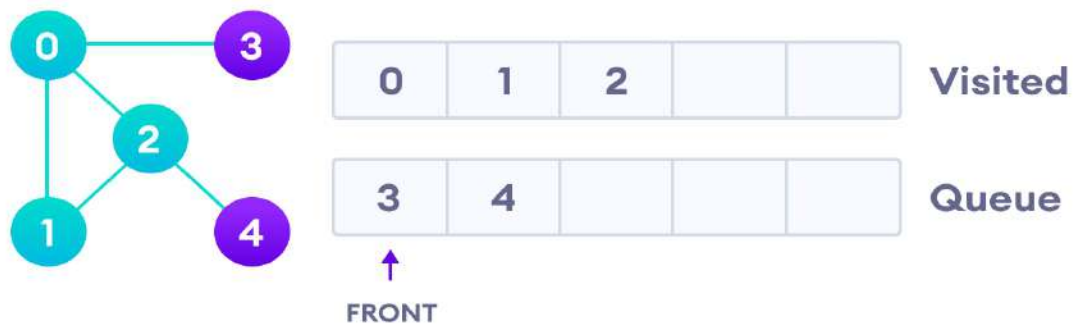
Visit start vertex and add its adjacent vertices to queue.

Next, we visit the element at the front of queue i.e., 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



Visit the first neighbour of start node 0, which is 1.

Vertex 2 has unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.

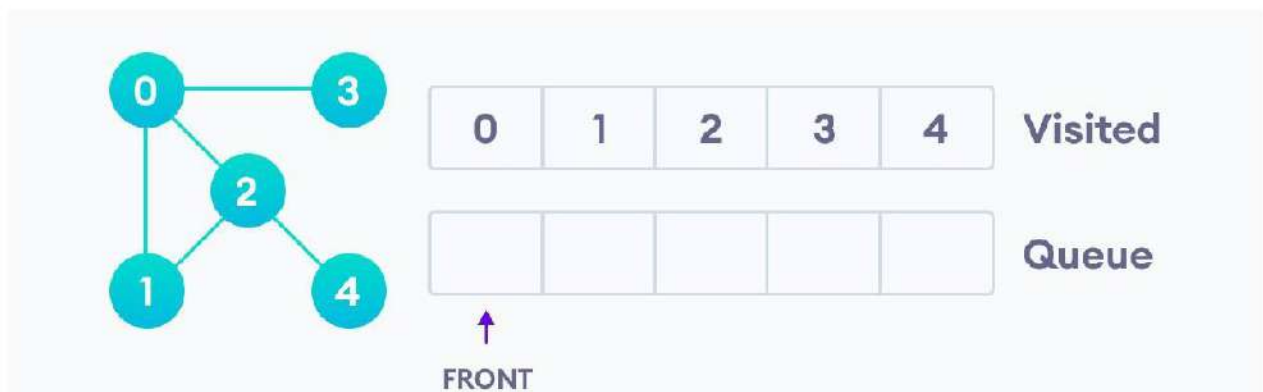


Visit 2 which was added to queue earlier to add its neighbours.



4 remains in the queue.

Only 4 remains in the queue since the only adjacent node of 3 i.e., 0 is already visited. We visit it.



Since the queue is empty, we have complete the Breadth First Traversal of the graph.

BFS pseudocode

```
Create a queue Q
mark v as visited and put v into Q
while Q is non empty
    remove the head u of Q
    mark and enqueue all (unvisited) neighbours of u
```

BFS Algorithm Complexity

The time complexity of the BFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is $O(V)$.

Conclusion:-

Thus, we have implemented Depth First Search algorithm and Breadth First Search algorithm. Using undirected graph and developed a recursive algorithm for searching all the vertices of a graph.

Sample Expert Viva-vice Questions:-

1. What is the difference between DFS and BFS?
2. Is BFS & DFS a complete algorithm? Is BFS & DFS optimal algorithm?
3. Why do we prefer queues instead of other data structures while implementing BFS?
4. Why can we not implement DFS using Queues? Why do we prefer stacks instead of other data structures?
5. Why can we not use DFS for finding shortest possible path?